

Day 11: Evaluating and Improving a RAG System

1. 5-line beginner summary

RAG evaluation checks whether the system retrieved the right documents and generated the right answer. A RAG answer is good only when it is relevant, grounded in retrieved context, and not hallucinated. Retrieval quality measures whether useful chunks were found. Answer quality measures whether the final response is correct, clear, and supported by sources. A strong RAG system improves continuously using metrics, human review, feedback, and better retrieval/prompting.

2. Why RAG evaluation is important

A RAG system has two major parts:

1. Retriever -> finds relevant chunks
2. Generator -> uses chunks to answer

If the answer is wrong, the problem may be in either part.

Example:

User asks:

What is the company maternity leave policy?

Possible failures:

Retriever failure:
Wrong chunks were retrieved, maybe from travel policy instead of HR policy.

Generator failure:
Correct chunks were retrieved, but the LLM misunderstood them.

Grounding failure:
The LLM answered from general knowledge instead of using company policy.

Hallucination:
The LLM invented a policy that does not exist.

So RAG evaluation is important because it helps answer:

Did we retrieve the right information?
Did the LLM use the retrieved information?
Is the final answer correct?
Is the answer useful for the user?
Can we trust this system in production?

3. Descriptive notes

3.1 Retrieval quality

Retrieval quality checks whether the retriever found useful context.

In RAG, retrieval happens before answer generation.

User question -> Retriever -> Top-k chunks -> LLM -> Answer

If the retriever fails, the LLM may not have enough information to answer correctly.

Example:

Question:

Can employees carry forward unused annual leave?

Good retrieved chunk:

Employees may carry forward up to 10 unused annual leave days to the next calendar year.

Bad retrieved chunk:

Employees must submit travel claims within 30 days.

Retrieval quality asks:

Were the retrieved chunks relevant?
Did we retrieve enough information?
Did we miss an important policy section?
Did we rank the best chunk near the top?

3.2 Answer quality

Answer quality checks the final response produced by the LLM.

A good answer should be:

Correct
Clear
Complete
Relevant
Grounded in context
Not hallucinated
Easy to understand

Question:

How many annual leave days can I carry forward?

Retrieved context:

Employees may carry forward up to 10 unused annual leave days.

Good answer:

You can carry forward up to 10 unused annual leave days.

Bad answer:

You can carry forward all unused leave days.

The bad answer sounds confident but is incorrect.

3.3 Groundedness

Groundedness means the answer is supported by the retrieved context.

A grounded answer does not go beyond the evidence.

Example context:

Employees can work from home up to 2 days per week with manager approval.

Grounded answer:

Employees can work from home up to 2 days per week, but they need manager approval.

Not grounded:

Employees can work from home permanently from any country.

The second answer is not supported by the context.

Groundedness answers:

Can we point to the source document for this answer?

3.4 Faithfulness

Faithfulness is very close to groundedness.

A faithful answer accurately represents the retrieved context.

Context:

Employees are eligible for parental leave after completing 12 months of service.

Faithful answer:

Employees become eligible for parental leave after 12 months of service.

Unfaithful answer:

Employees become eligible for parental leave immediately after joining.

The answer is unfaithful because it contradicts the context.

Simple difference:

Concept	Meaning
Groundedness	Is the answer supported by context?
Faithfulness	Does the answer correctly reflect the context?

3.5 Relevance

Relevance checks whether the answer actually addresses the user's question.

Question:

What is the reimbursement limit for internet bills?

Relevant answer:

The monthly internet reimbursement limit is ₹1,500.

Irrelevant answer:

Employees can claim travel reimbursement after approval.

The second answer may be true, but it does not answer the question.

Relevance applies to both:

Retrieved chunks
Generated answer

3.6 Hallucination detection

Hallucination means the LLM generates information that is not supported by the retrieved documents.

Example:

Context:

The policy allows reimbursement for laptop accessories up to ₹5,000.

Hallucinated answer:

Employees can claim ₹10,000 for laptop accessories and mobile phones.

The model added unsupported information.

Hallucination detection checks:

Is every claim supported by retrieved context?
Are there numbers, names, rules, dates, or conditions not present in the source?

Does the answer contradict the source?
Is the model guessing when context is missing?

A good RAG system should say:

I could not find this information in the provided policy documents.

instead of guessing.

3.7 Context precision

Context precision measures how much of the retrieved context is actually useful.

Suppose top 5 chunks are retrieved:

Chunk 1: Annual leave policy	Relevant
Chunk 2: Leave carry-forward rule	Relevant
Chunk 3: Travel policy	Not relevant
Chunk 4: Payroll policy	Not relevant
Chunk 5: Leave approval process	Relevant

Relevant chunks = 3 Total retrieved chunks = 5

Context precision = $3 / 5 = 0.60$

Higher context precision means the retriever is not adding too much noise.

Why this matters:

Too many irrelevant chunks confuse the LLM.
They increase cost.
They increase hallucination risk.
They reduce answer quality.

3.8 Context recall

Context recall measures whether the retriever found all important information needed to answer.

Suppose the correct answer requires 3 policy chunks:

Required Chunk A: Eligibility
Required Chunk B: Limit
Required Chunk C: Approval process

Retriever found:

Chunk A
Chunk B

It missed:

Chunk C

So:

Context recall = $2 / 3 = 0.67$

High recall means the system does not miss important evidence.

Simple difference:

Metric	Main question
Context precision	Are retrieved chunks mostly useful?
Context recall	Did we retrieve all needed chunks?

3.9 Human evaluation

Human evaluation means real people review RAG outputs.

Human reviewers check:

Is the answer correct? Is it easy to understand? Is it supported by the source? Is the answer complete? Would this be acceptable in production?

Human evaluation is especially important for:

HR policy assistant Legal document assistant Healthcare assistant Finance assistant Insurance assistant Enterprise knowledge assistant

Because automatic metrics may not catch everything.

Example human rating form:

Question: What is the probation period? Answer: The probation period is 6 months. Rate: Correctness: 5/5 Groundedness: 5/5 Completeness: 4/5 Clarity: 5/5 Risk: Low
--

3.10 Golden question-answer dataset

A golden dataset is a trusted test set used to evaluate the RAG system.

It contains:

```
User question
Expected answer
Expected source document
Expected relevant chunks
Important facts that must appear
Invalid facts that must not appear
```

Example:

```
Question:
How many days of annual leave can an employee carry forward?

Expected answer:
Employees can carry forward up to 10 unused annual leave days.

Expected source:
HR Leave Policy, Section 4.2

Expected relevant chunk:
"Employees may carry forward up to 10 unused annual leave days to the next
calendar year."
```

Why golden datasets are useful:

```
They make evaluation repeatable.
They help compare different RAG versions.
They help detect regression.
They help tune chunking, retrieval, reranking, and prompts.
```

3.11 Offline evaluation

Offline evaluation means testing the RAG system before real users use it.

You run test questions from the golden dataset and measure quality.

Offline evaluation helps compare:

```
Chunk size 500 vs 1000
Top-k 5 vs Top-k 10
Vector search vs hybrid search
With reranker vs without reranker
Old prompt vs new prompt
Embedding model A vs embedding model B
```

Example:

```
Version A:
Context precision = 0.62
Faithfulness = 0.78
Answer relevance = 0.81

Version B:
```

```
Context precision = 0.76
Faithfulness = 0.89
Answer relevance = 0.87
```

Version B is likely better.

Offline evaluation is done before production deployment.

3.12 Online feedback

Online feedback means learning from real users after deployment.

Examples:

```
Thumbs up / thumbs down
User comments
Did the user ask a follow-up?
Did the user reformulate the same question?
Did the user click the source citation?
Did the user escalate to human support?
```

Online feedback shows real-world quality.

Example:

```
User question:
Can I take unpaid leave during probation?

System answer:
Unclear answer

User feedback:
Thumbs down
Comment: "This does not answer probation-specific leave."
```

This feedback can become a new golden test case.

3.13 Continuous improvement loop

RAG evaluation is not a one-time activity.

A production RAG system should continuously improve.

Loop:

```
Collect questions
Evaluate responses
Find failure type
Improve retrieval or prompt
Test offline
Deploy safely
Monitor online feedback
Repeat
```


Common improvement areas:

Better document parsing
Better chunking
Metadata filtering
Hybrid search
Reranking
Query rewriting
Better prompt
Better fallback response
Human review for risky questions

4. Easy example: Internal policy assistant

Imagine a company builds an internal policy assistant.

Employees ask questions like:

How many work-from-home days are allowed?
What is the laptop reimbursement limit?
Can I carry forward unused leave?
What is the notice period?
Do I need approval for business travel?

Documents:

HR Policy
Leave Policy
Travel Policy
Expense Policy
Remote Work Policy
IT Asset Policy

Question:

Can I work from home 3 days per week?

Retrieved chunks:

Chunk 1:
Employees can work from home up to 2 days per week with manager approval.

Chunk 2:
Employees must be available during core working hours.

Chunk 3:
Travel claims must be submitted within 30 days.

Generated answer:

No, the policy allows work from home up to 2 days per week with manager approval.

Evaluation:

Retrieval quality:

Chunk 1 is highly relevant.

Chunk 2 is partially relevant.

Chunk 3 is irrelevant.

Answer quality:

The answer is correct.

Groundedness:

The answer is supported by Chunk 1.

Faithfulness:

The answer correctly says 2 days, not 3 days.

Relevance:

The answer directly answers the user question.

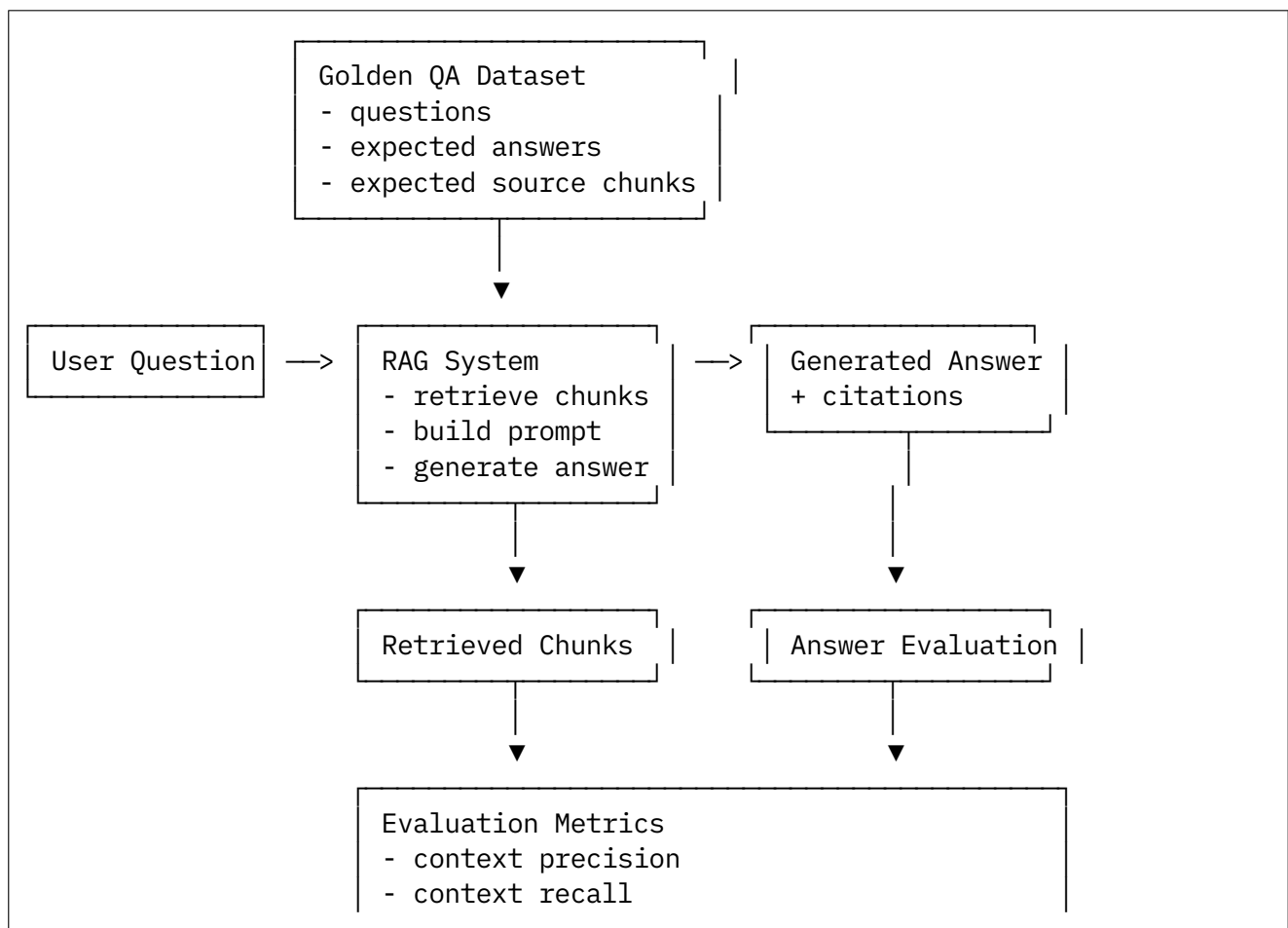
Hallucination:

No hallucination.

Improvement:

Remove irrelevant travel policy chunks using metadata filtering or reranking.

5. ASCII diagram: RAG evaluation flow



- answer relevance
- groundedness
- faithfulness
- hallucination rate



- Improvement Actions
- improve chunking
 - tune top-k
 - add hybrid search
 - add reranking
 - rewrite prompt
 - update golden dataset

6. Pseudocode for evaluating RAG responses

```
FUNCTION evaluate_rag_system(golden_dataset):

    results = []

    FOR each test_case IN golden_dataset:

        question = test_case.question
        expected_answer = test_case.expected_answer
        expected_chunks = test_case.expected_relevant_chunks

        rag_output = run_rag_pipeline(question)

        retrieved_chunks = rag_output.retrieved_chunks
        generated_answer = rag_output.answer

        retrieval_score = evaluate_retrieval(
            retrieved_chunks,
            expected_chunks
        )

        answer_score = evaluate_answer(
            generated_answer,
            expected_answer
        )

        groundedness_score = check_groundedness(
            generated_answer,
            retrieved_chunks
        )

        faithfulness_score = check_faithfulness(
            generated_answer,
            retrieved_chunks
```

```

    )

    hallucination_score = detect_hallucination(
        generated_answer,
        retrieved_chunks
    )

    relevance_score = check_relevance(
        question,
        generated_answer
    )

    result = {
        "question": question,
        "retrieval_score": retrieval_score,
        "answer_score": answer_score,
        "groundedness_score": groundedness_score,
        "faithfulness_score": faithfulness_score,
        "hallucination_score": hallucination_score,
        "relevance_score": relevance_score
    }

    results.append(result)

final_report = aggregate_results(results)

RETURN final_report

```

More practical pseudocode

```

FUNCTION evaluate_retrieval(retrieved_chunks, expected_chunks):

    relevant_count = 0

    FOR chunk IN retrieved_chunks:
        IF chunk matches any expected_chunk:
            relevant_count = relevant_count + 1

    context_precision = relevant_count / total_retrieved_chunks

    found_expected_count = 0

    FOR expected_chunk IN expected_chunks:
        IF expected_chunk found in retrieved_chunks:
            found_expected_count = found_expected_count + 1

    context_recall = found_expected_count / total_expected_chunks

    RETURN context_precision, context_recall

```

```

FUNCTION check_groundness(answer, retrieved_chunks):

```

```

claims = extract_claims(answer)

supported_claims = 0

FOR claim IN claims:
    IF claim is supported by retrieved_chunks:
        supported_claims = supported_claims + 1

groundedness = supported_claims / total_claims

RETURN groundedness

```

```

FUNCTION detect_hallucination(answer, retrieved_chunks):

    claims = extract_claims(answer)

    unsupported_claims = 0

    FOR claim IN claims:
        IF claim is not supported by retrieved_chunks:
            unsupported_claims = unsupported_claims + 1

    hallucination_rate = unsupported_claims / total_claims

    RETURN hallucination_rate

```

```

FUNCTION continuous_improvement_loop():

    collect_user_questions()

    create_or_update_golden_dataset()

    run_offline_evaluation()

    identify_failure_patterns()

    IF retrieval_quality_is_low:
        improve_chunking()
        improve_metadata_filtering()
        try_hybrid_search()
        add_reranking()

    IF answer_quality_is_low:
        improve_prompt()
        add_grounding_instruction()
        improve_citation_requirement()
        reduce_temperature()

    IF hallucination_is_high:
        add_strict_fallback_rule()
        require_answer_from_context_only()
        improve_context_quality()

```

```

deploy_new_version_safely()

monitor_online_feedback()

repeat()

```

7. Simple metrics table

Metric	What it checks	Simple meaning	Example question
Retrieval precision	How many retrieved chunks are useful	Did we avoid noisy chunks?	Out of top 5 chunks, how many are relevant?
Retrieval recall	Whether all needed chunks were found	Did we miss important evidence?	Did we retrieve the policy limit and eligibility rule?
Context precision	Usefulness of retrieved context	Is the context clean?	Are irrelevant chunks included?
Context recall	Completeness of retrieved context	Is enough evidence retrieved?	Did we retrieve all required policy sections?
Answer relevance	Whether answer addresses the question	Did we answer what user asked?	User asked about leave, answer should not discuss travel
Groundedness	Whether answer is supported by context	Can we point to source evidence?	Is the answer backed by retrieved chunks?
Faithfulness	Whether answer correctly represents context	Did the LLM distort the source?	Source says 10 days, answer should not say 15
Hallucination rate	Unsupported claims in answer	Did the model invent anything?	Did it add rules not present in policy?
Citation accuracy	Whether citations point to correct source	Are references trustworthy?	Does the cited chunk actually support the sentence?
Human score	Human judgment of quality	Would a reviewer approve it?	Is it correct, clear, complete, and safe?
User feedback score	Real user satisfaction	Did users find it helpful?	Thumbs up/down, comments, escalations
Latency	Response time	Is the system fast enough?	Did the answer arrive within acceptable time?
Cost per query	Compute/API cost	Is it affordable?	Are we retrieving or generating too much?

8. How to improve a RAG system based on evaluation

Problem 1: Low context precision

Symptom:

Retriever brings many irrelevant chunks.

Example:

Question: What is the leave carry-forward limit?

Retrieved:

- Leave policy
- Travel policy
- Expense policy
- Office parking policy

Fix:

Use metadata filtering
Use hybrid search
Use reranking
Improve chunking
Reduce top-k
Use better query rewriting

Problem 2: Low context recall

Symptom:

Retriever misses important chunks.

Example:

Answer requires:

Eligibility rule
Limit rule
Approval rule

But retriever only finds:

Limit rule

Fix:

Increase top-k
Use multi-query retrieval
Use parent-child retrieval
Use better embeddings
Use document hierarchy
Use hybrid search

Problem 3: Low groundedness

Symptom:

Answer includes claims not found in context.

Fix:

Prompt the model to answer only from context
Ask model to cite each claim
Add fallback: "I could not find this in the documents"
Use lower temperature
Use better retrieved chunks

Problem 4: Low answer relevance

Symptom:

Answer is true but does not answer the user question.

Fix:

Improve query understanding
Rewrite ambiguous user questions
Improve prompt instruction
Use conversation history carefully
Evaluate answer against original question

Problem 5: High hallucination

Symptom:

Model invents numbers, rules, policy details, or exceptions.

Fix:

Use stricter grounding prompt
Use citation-based answer generation
Reject unsupported claims
Improve context compression
Add human review for high-risk answers

9. Prompt optimization for RAG evaluation and improvement

A basic prompt may say:

Answer the question using the context.

A stronger RAG prompt says:

You are an internal policy assistant.

Use only the provided context to answer.
If the answer is not present in the context, say:
"I could not find this information in the provided policy documents."

Do not guess.
Do not use outside knowledge.
Mention the policy section if available.
Keep the answer concise and clear.

Context:
{retrieved_context}

Question:
{user_question}

Why this helps:

It reduces hallucination.
It improves groundedness.
It forces the model to admit missing context.
It improves trust.

10. Golden dataset example for internal policy assistant

Question	Expected answer	Expected source
How many annual leave days can I carry forward?	Up to 10 unused annual leave days	Leave Policy Section 4.2
Can I work from home 3 days per week?	No, policy allows up to 2 days per week with manager approval	Remote Work Policy Section 2.1
What is the laptop reimbursement limit?	₹60,000 with manager and finance approval	IT Asset Policy Section 5.3
When should travel claims be submitted?	Within 30 days of travel completion	Travel Policy Section 6.1
Can I claim internet reimbursement?	Yes, up to ₹1,500 per month if approved	Expense Policy Section 3.4

Golden datasets should include different question types:

Simple factual questions
Multi-document questions
Policy exception questions
Ambiguous questions
Questions where answer is missing
Questions with outdated policy risk
Questions requiring citations

Very important: include questions where the answer is **not available** in the documents.

This tests whether the system can avoid hallucination.

11. Offline evaluation example

Suppose we test 100 golden questions.

Version 1 results:

```
Context precision: 65%
Context recall: 70%
Answer relevance: 78%
Groundedness: 72%
Hallucination rate: 18%
```

After improvement:

```
Added hybrid search
Added reranking
Improved prompt
Added fallback rule
```

Version 2 results:

```
Context precision: 82%
Context recall: 84%
Answer relevance: 89%
Groundedness: 91%
Hallucination rate: 6%
```

This means Version 2 is clearly better.

12. Online feedback example

After deployment, employees use the policy assistant.

Feedback collected:

```
Question: Can I work from home from another country?
Answer: Yes, remote work is allowed up to 2 days per week.
User feedback: Thumbs down
Comment: "This does not answer international remote work."
```

Analysis:

```
The answer was partially relevant but incomplete.
The system confused normal work-from-home with international remote work.
```

Improvement:

```
Add golden question about international remote work.
Improve retrieval to search global mobility policy.
Add prompt rule to clarify when policy type is different.
```

13. Continuous improvement loop

```
Step 1: Collect real user questions
Step 2: Identify bad answers
Step 3: Label failure type
Step 4: Add cases to golden dataset
Step 5: Improve retrieval, chunking, reranking, or prompt
Step 6: Run offline evaluation
Step 7: Compare metrics with previous version
Step 8: Deploy improved version
Step 9: Monitor online feedback
Step 10: Repeat
```

Failure types:

```
Wrong document retrieved
Relevant document missed
Answer not grounded
Answer too vague
Answer too long
Answer hallucinated
Answer missed important condition
Citation incorrect
User question ambiguous
Policy document outdated
```

14. Common mistakes

Mistake 1: Evaluating only the final answer

Many teams check only whether the answer looks good.

But RAG needs separate evaluation for:

```
Retriever
Context
Generator
Citations
User feedback
```

Mistake 2: Ignoring retrieval metrics

If retrieval is poor, the LLM cannot produce a reliable answer.

Bad retrieval usually causes:

```
Wrong answer
Incomplete answer
Hallucination
Poor citation
Low user trust
```

Mistake 3: Using only generic questions

A golden dataset should not contain only easy questions.

Include:

Simple questions
Complex questions
Missing-answer questions
Multi-policy questions
Ambiguous questions
Exception-based questions

Mistake 4: Not testing missing information

A good RAG system must know when to say:

I could not find this information in the provided documents.

If you do not test this, the model may confidently hallucinate.

Mistake 5: Thinking high similarity score means correct answer

A chunk may have high vector similarity but still not answer the question.

Example:

Question:

Can I carry forward leave?

Retrieved chunk:

Employees can apply for leave using the HR portal.

This is related to leave, but it does not answer carry-forward.

Mistake 6: Not checking citations

A RAG answer with citations is not automatically trustworthy.

You must check:

Does the citation actually support the answer?
Is the cited chunk relevant?
Did the answer use unsupported information?

Mistake 7: Too much context

More context is not always better.

Too much context can:

Confuse the model
Increase cost
Increase latency
Add irrelevant information
Reduce answer accuracy

Mistake 8: No human review

Automatic evaluation is useful, but human review is still needed for sensitive use cases.

Especially for:

HR
Legal
Finance
Healthcare
Compliance
Insurance

Mistake 9: Not tracking version changes

Always track:

Embedding model version
Chunking strategy
Retriever type
Reranker version
Prompt version
LLM version
Golden dataset version
Evaluation score

Without version tracking, you cannot know what improved or broke the system.

Mistake 10: Not using production feedback

Offline evaluation is not enough.

Real users ask unexpected questions.

Production feedback helps discover:

New question patterns
Missing documents
Confusing policies
Poor answers
Outdated information

15. Final beginner-friendly mental model

Think of RAG evaluation like checking a student's open-book exam.

Retriever = Did the student open the right book pages?

Generator = Did the student write the correct answer?

Groundedness = Is the answer supported by the book?

Faithfulness = Did the student explain the book correctly?

Relevance = Did the student answer the actual question?

Hallucination = Did the student invent something not in the book?

Golden dataset = Previous exam questions with correct answers.

Offline evaluation = Practice test before real exam.

Online feedback = Real user feedback after deployment.

Continuous improvement = Keep fixing weak areas.

A strong enterprise RAG system is not built only by connecting documents to an LLM. It becomes reliable when retrieval, grounding, answer quality, feedback, and continuous evaluation are all measured and improved.